

# Timestep sensitivities: CTSE and OTI real matrices

## Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1 Background</b>                                        | <b>1</b>  |
| 1.1 Automatic differentiation                              | 1         |
| 1.2 Finite differences and their limitations               | 1         |
| 1.3 First derivatives with CTSE                            | 2         |
| 1.3.1 Worked example: $f(t) = \exp(t^2)$                   | 3         |
| 1.4 OTI arithmetic and Taylor coefficients                 | 5         |
| 1.4.1 Worked example: $f(t) = \exp(t^2)$                   | 6         |
| 1.5 Cauchy–Riemann matrix representation                   | 7         |
| 1.5.1 Univariate functions in the representation           | 7         |
| 1.5.2 Worked example: $f(t) = \exp(t^2)$                   | 8         |
| 1.5.3 Matrices of OTI numbers                              | 8         |
| <b>2 Implementation</b>                                    | <b>9</b>  |
| 2.1 Original response calculation                          | 9         |
| 2.2 CTSE: complex input and derivative extraction          | 9         |
| 2.3 OTI–CR: real block assembly and coefficient extraction | 10        |
| 2.4 Comparison and verification code                       | 11        |
| <b>3 Results</b>                                           | <b>12</b> |
| 3.1 Function values and derivative agreement               | 12        |
| 3.2 Derivative profiles and step-size dependence           | 13        |

## 1 Background

This section introduces differentiation methods for a generic scalar or vector-valued function  $F(t)$  of one real parameter. Section 2 applies these methods to the repository by identifying the changes to the original numerical calculation. Section 3 presents the comparisons.

### 1.1 Automatic differentiation

Automatic differentiation (AD), also known as *algorithmic differentiation* or *autodiff*, is a technique for computing numerical derivatives of functions expressed as computer programs with high accuracy. AD is used in machine learning, optimization, and scientific computing, where efficient and accurate gradients are required [1, 2]. It is particularly useful when a calculation is too complex for practical manual differentiation or when an explicit derivative formula is unavailable.

AD decomposes a function evaluation into arithmetic operations and elementary functions with known derivatives, forming an *evaluation trace*. This trace can be represented as a computational graph: nodes describe operations and intermediate values, while directed edges describe their dependencies. Applying the chain rule along these dependencies produces derivatives of the complete calculation [3]. For example, the evaluation of  $f(t) = \sin(t^2)$  can be written as  $v_1 = t^2$  followed by  $v_2 = \sin(v_1)$ . Propagating derivatives gives

$$\frac{dv_1}{dt} = 2t, \quad \frac{dv_2}{dt} = \cos(v_1) \frac{dv_1}{dt} = 2t \cos(t^2).$$

The derivative is evaluated numerically without constructing a global symbolic expression or approximating a limit with a finite perturbation. For a differentiable evaluation trace, AD introduces no finite-difference truncation error, although floating-point rounding and conditioning still affect accuracy. Forward mode propagates derivatives from inputs to outputs; reverse mode propagates sensitivities from outputs back to inputs [3].

## 1.2 Finite differences and their limitations

Where AD propagates derivatives through the evaluation trace, finite differences (FD) approximate them from function values at nearby real inputs. For a sufficiently smooth function  $F$  and a real step  $h$ ,

$$F'(t) \approx \frac{F(t+h) - F(t)}{h}, \quad F'(t) \approx \frac{F(t+h) - F(t-h)}{2h}. \quad (1)$$

The forward and centered formulas in Equation (1) have truncation errors of order  $h$  and  $h^2$ , respectively. Reducing  $h$  reduces these errors, but eventually the subtraction of nearly equal function values amplifies roundoff relative to the small difference. Noise or solver errors in function values are also amplified by division by  $h$ . Consequently, a smaller step does not always give a more accurate derivative [4]. Higher-order FD formulas amplify evaluation errors through division by higher powers of  $h$ , and suitable steps depend on the derivative order and the function's scales. FD requires only function evaluations, making it a useful independent check, but its accuracy depends on the stencil and step-size choice.

**Truncation and roundoff in a worked example.** Figure 1 illustrates this tradeoff for  $f(t) = \exp(t^2)$  at  $t = 1$ , the same example used in Sections 1.3.1, 1.4.1, and 1.5.2. For the centered formula in Equation (1),

$$D_c(h) = \frac{\exp((1+h)^2) - \exp((1-h)^2)}{2h} = 2e + \frac{10}{3}eh^2 + O(h^4). \quad (2)$$

Thus the leading truncation error is  $(10/3)eh^2$ . A simple illustrative scale for roundoff is  $\epsilon_{\text{mach}}e/h$ , where  $\epsilon_{\text{mach}}$  is the spacing from 1 to the next larger `Float64` number. Adding the two scales gives the familiar U-shaped error model. The roundoff scale is a heuristic, not a measured error or a rigorous bound: actual errors depend on rounding in the inputs, function evaluations, subtraction, and division.

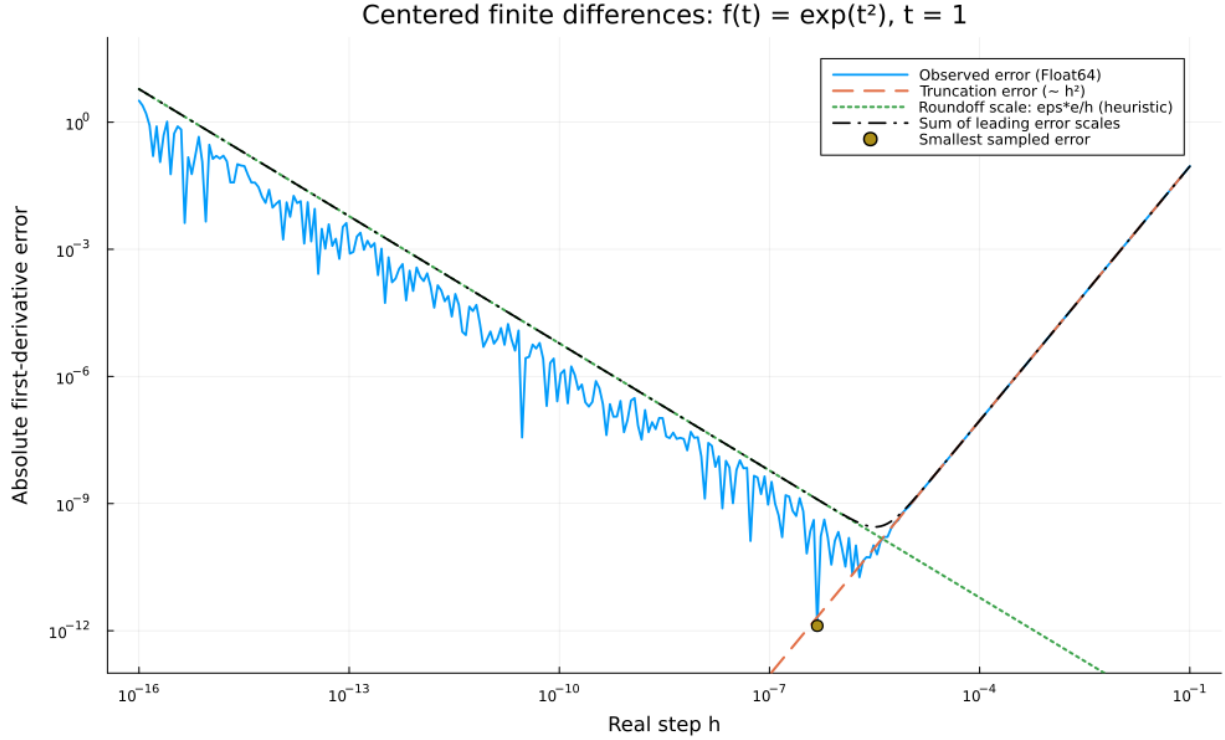


Figure 1: Centered finite-difference error for  $f(t) = \exp(t^2)$  at  $t = 1$ . The observed error compares the `Float64` derivative with a 256-bit reference  $2e$ . Truncation error is evaluated with the same centered stencil in 256-bit arithmetic. The roundoff scale and summed error model illustrate the competing trends; they are not bounds on the observed error.

At large  $h$ , truncation dominates and the error decreases approximately as  $h^2$  when the step is reduced. At small  $h$ , cancellation and rounding dominate, producing an irregular increase in error. The smallest observed error lies between these regimes; its location depends on the function and arithmetic. Figure 2 contrasts this behavior with CTSE for the same function and evaluation point. The figure and raw error data can be regenerated from the repository root:

```
julia --project=. docs/fd_worked_example.jl
```

### 1.3 First derivatives with CTSE

Complex Taylor series expansion (CTSE), also called the complex-step method, evaluates the response at a complex input  $t + ih$ , where  $i^2 = -1$  and  $h > 0$  is a small real step [5]. If  $F$  is analytic and real-valued on the real axis,

$$F(t + ih) = F(t) + ihF'(t) - \frac{h^2}{2}F''(t) - \frac{ih^3}{6}F'''(t) + O(h^4).$$

Taking the imaginary part and dividing by  $h$  gives

$$\boxed{F'(t) \approx \frac{\text{Im } F(t + ih)}{h}}, \quad \frac{\text{Im } F(t + ih)}{h} = F'(t) - \frac{h^2}{6}F'''(t) + O(h^4). \quad (3)$$

The real part supplies  $F(t)$  with an  $O(h^2)$  error. Unlike a finite difference, Equation (3) does not subtract nearly equal response values, allowing a very small  $h$ . CTSE is a numerical derivative approximation; OTI arithmetic, introduced in Section 1.4, propagates Taylor coefficients algebraically.

The evaluated function must admit an analytic extension near the real input. Operations such as absolute values, conjugation, or premature removal of imaginary components can invalidate the method. Standard CTSE provides the first derivative in Equation (3); one evaluation does not separate the higher derivatives also present in its Taylor expansion. Other complex-step extensions exist, but are outside the scope of this comparison.

### 1.3.1 Worked example: $f(t) = \exp(t^2)$

Consider the composition  $v = t^2$ ,  $f = \exp(v)$  at  $t_0 = 1$ . Sections 1.4.1 and 1.5.2 evaluate this same function using OTI arithmetic and its CR representation. Direct differentiation supplies reference values:

$$f(1) = e, \quad f'(1) = 2e, \quad f''(1) = 6e. \quad (4)$$

Here  $e = \exp(1)$  is Euler's number. For CTSE, first seed the input as  $\tilde{t} = 1 + ih$ . Then evaluate the original two operations with complex arithmetic, retaining the imaginary component at each step:

$$\begin{aligned} \tilde{v} &= (1 + ih)^2 = 1 - h^2 + 2ih, \\ \tilde{f} &= \exp(\tilde{v}) = \exp(1 - h^2) [\cos(2h) + i \sin(2h)]. \end{aligned}$$

The real and imaginary parts therefore give

$$f(1) \approx \exp(1 - h^2) \cos(2h), \quad f'(1) \approx \exp(1 - h^2) \frac{\sin(2h)}{h}. \quad (5)$$

Expanding the extracted derivative in Equation (5) gives

$$\exp(1 - h^2) \frac{\sin(2h)}{h} = 2e - \frac{10}{3}eh^2 + O(h^4). \quad (6)$$

The leading term matches Equation (2) in magnitude and differs only in sign. This is not particular to the example: for any analytic  $F$ , the centered difference carries  $+h^2 F'''(t)/6$  and the complex step  $-h^2 F'''(t)/6$ . The two methods therefore share the same truncation error, and what separates them is the other error source. The centered difference forms  $F(t + h) - F(t - h)$ , a subtraction of increasingly close values, so reducing  $h$  eventually destroys accuracy through cancellation. The complex step takes an imaginary part rather than a difference, so no such subtraction occurs and  $h$  may be reduced far below the finite-difference optimum.

Equation (6) predicts second-order convergence as  $h$  decreases, until numerical rounding dominates. Table 1 presents the computed function values and first derivatives for selected steps. Figure 2 shows the corresponding convergence and the leading truncation-error magnitude  $(10/3)eh^2$ . The original operations are evaluated once per complex input; neither a symbolic derivative nor a difference of nearby function values is needed.

**Numerical results for the worked example.** The evaluations use Julia’s ordinary `Float64` complex arithmetic. The reference values are  $f(1) = e \approx 2.718281828459$  and  $f'(1) = 2e \approx 5.436563656918$ . To distinguish rounding error from truncation error, absolute errors are computed against a 256-bit evaluation of  $2e$ , using the full stored derivative rather than its printed digits.

Table 1: CTSE evaluations of  $f(t) = \exp(t^2)$  at  $t = 1$ . Function values and derivatives are printed to twelve decimal places; errors use the full stored values and a high-precision reference.

| Step $h$                | $\operatorname{Re} \tilde{f}$ | $\operatorname{Im} \tilde{f}/h$ | Absolute derivative error |
|-------------------------|-------------------------------|---------------------------------|---------------------------|
| $1.000 \times 10^{-1}$  | 2.637588959484                | 5.346657516342                  | $8.991 \times 10^{-2}$    |
| $1.000 \times 10^{-2}$  | 2.717466429984                | 5.435657633647                  | $9.060 \times 10^{-4}$    |
| $1.000 \times 10^{-3}$  | 2.718273673622                | 5.436554595986                  | $9.061 \times 10^{-6}$    |
| $1.000 \times 10^{-4}$  | 2.718281746911                | 5.436563566309                  | $9.061 \times 10^{-8}$    |
| $1.000 \times 10^{-6}$  | 2.718281828451                | 5.436563656909                  | $9.061 \times 10^{-12}$   |
| $1.000 \times 10^{-8}$  | 2.718281828459                | 5.436563656918                  | $2.891 \times 10^{-16}$   |
| $1.000 \times 10^{-12}$ | 2.718281828459                | 5.436563656918                  | $2.891 \times 10^{-16}$   |
| $1.000 \times 10^{-20}$ | 2.718281828459                | 5.436563656918                  | $2.891 \times 10^{-16}$   |

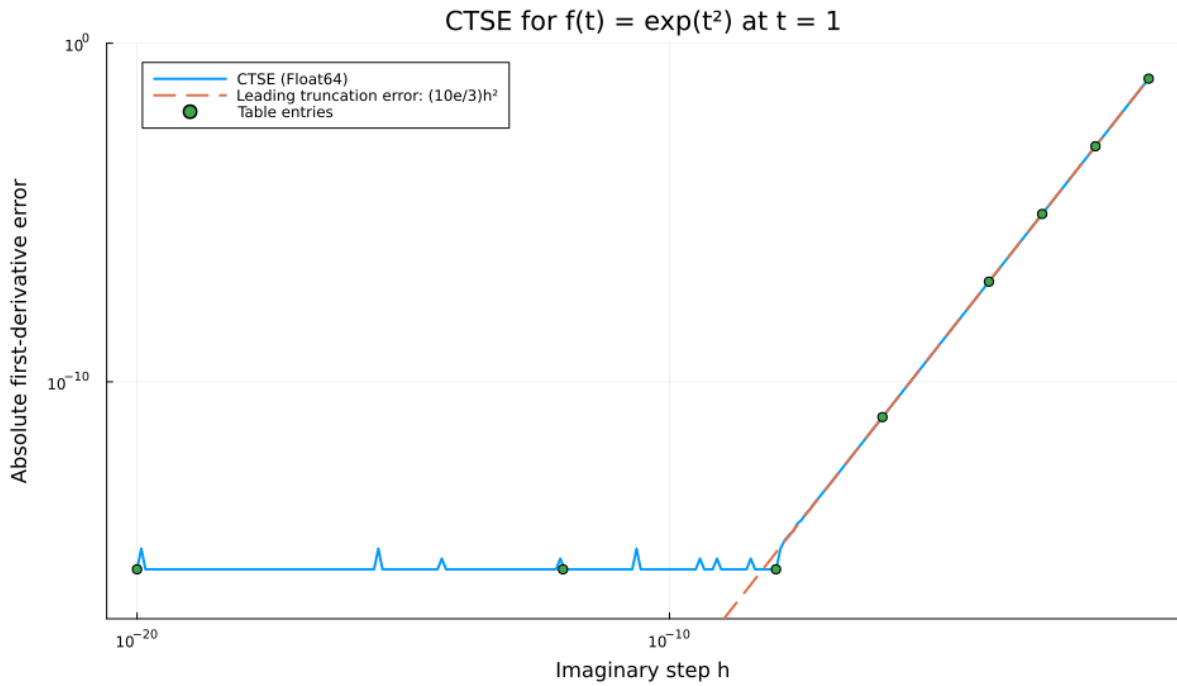


Figure 2: CTSE first-derivative convergence for the worked example. The dashed line is the leading truncation error from Equation (6); markers identify the steps in Table 1.

Figure 2 shows the error initially decreasing in proportion to  $h^2$ . For sufficiently small steps, the error reaches a floating-point rounding level and no longer decreases systematically. Unlike real finite differences, reducing  $h$  further does not introduce subtractive cancellation in the derivative extraction for this example. The table, figure, and raw CSV data can be regenerated from the repository root:

```
julia --project=. docs/ctse_worked_example.jl
```

## 1.4 OTI arithmetic and Taylor coefficients

For one variable and truncation order  $p$ , consider two order-truncated imaginary (OTI) numbers [6]:

$$a = \sum_{j=0}^p a_j \varepsilon^j, \quad b = \sum_{j=0}^p b_j \varepsilon^j, \quad a_j, b_j \in \mathbb{R}, \quad \varepsilon^{p+1} = 0. \quad (7)$$

The symbol  $\varepsilon$  is *nilpotent*: sufficiently high powers vanish. It is neither a small floating-point number nor the usual complex unit  $i$  with  $i^2 = -1$ .

At  $p = 1$ , Equation (7) reduces to the *dual numbers*  $a_0 + a_1 \varepsilon$  with  $\varepsilon^2 = 0$ , which carry a value and one derivative through a forward-mode evaluation. OTI numbers extend that same construction to order  $p$ , carrying  $p$  derivatives at once.

The prescribed square is also what separates this from Section 1.3. Under the complex step,  $i^2 = -1$  folds the third-order Taylor term back into the imaginary part, which is the origin of the  $O(h^2)$  truncation error in Equation (3). Nilpotency annihilates that term instead, so the retained coefficients are exact in the algebra rather than approximations improved by shrinking a step. The name retains *imaginary* because the construction parallels  $\mathbb{C}$ , adjoining to the reals a new unit with a prescribed square, and because extracting a coefficient plays the role of taking an imaginary part, as the notation  $\text{Im}_{\varepsilon^k}$  in Equation (11) reflects.

Addition combines coefficients of the same power:

$$a + b = \sum_{j=0}^p (a_j + b_j) \varepsilon^j. \quad (8)$$

At order two, Equation (8) becomes

$$a + b = (a_0 + b_0) + (a_1 + b_1) \varepsilon + (a_2 + b_2) \varepsilon^2.$$

Multiplication collects terms of equal degree and discards terms above order  $p$ :

$$ab = \sum_{k=0}^p \left( \sum_{j=0}^k a_j b_{k-j} \right) \varepsilon^k.$$

For example, at order two,  $a = a_0 + a_1 \varepsilon + a_2 \varepsilon^2$  and  $b = b_0 + b_1 \varepsilon + b_2 \varepsilon^2$  give

$$ab = a_0 b_0 + (a_0 b_1 + a_1 b_0) \varepsilon + (a_0 b_2 + a_1 b_1 + a_2 b_0) \varepsilon^2.$$

The coefficient  $a_0 b_1 + a_1 b_0$  reproduces the first-derivative product rule when the inputs carry Taylor coefficients.

An OTI scalar type can support *operator overloading*: operations such as addition and multiplication are defined to combine coefficients according to these rules. Elementary functions such as  $\exp$  and  $\sin$  can likewise be overloaded using their Taylor expansions. For  $a = a_0 + \sum_{r=1}^p a_r \varepsilon^r$ , the expansion about its real part  $a_0$  is evaluated at the full OTI input as

$$g\left(a_0 + \sum_{r=1}^p a_r \varepsilon^r\right) = \sum_{j=0}^p \frac{g^{(j)}(a_0)}{j!} \left(\sum_{r=1}^p a_r \varepsilon^r\right)^j. \quad (9)$$

Powers above  $p$  vanish because the displacement has zero constant coefficient and  $\varepsilon^{p+1} = 0$ . For example, at order two,

$$\exp(a_0 + a_1 \varepsilon + a_2 \varepsilon^2) = \exp(a_0) \left[ 1 + (a_1 \varepsilon + a_2 \varepsilon^2) + \frac{(a_1 \varepsilon + a_2 \varepsilon^2)^2}{2} \right].$$

A calculation composed of supported operations then propagates coefficients through the same sequence of function evaluations. Section 1.5 provides an equivalent real matrix representation; the implementation in Section 2.3 uses that representation rather than defining an overloaded OTI scalar type.

Seeding the input as  $t + \varepsilon$  gives, for an analytic function,

$$F(t + \varepsilon) = F(t) + F'(t) \varepsilon + \frac{F''(t)}{2!} \varepsilon^2 + \cdots + \frac{F^{(p)}(t)}{p!} \varepsilon^p. \quad (10)$$

Define  $\text{Im}_{\varepsilon^k}(a) := a_k$  to mean extraction of the coefficient of  $\varepsilon^k$ ; this is not the ordinary complex imaginary-part operator. Equation (10) therefore yields the unscaled derivative

$$\boxed{F^{(k)}(t) = k! \text{Im}_{\varepsilon^k}(F(t + \varepsilon))}, \quad 1 \leq k \leq p. \quad (11)$$

The constant coefficient is  $F(t)$ . The unit seed in  $t + \varepsilon$  fixes the scaling in Equation (11); no numerical perturbation step is required. For example,  $(t + \varepsilon)^2 = t^2 + 2t\varepsilon + \varepsilon^2$ , so the second derivative is  $2! \times 1 = 2$ . Truncation does not approximate the retained Taylor coefficients in exact arithmetic; numerical computations still incur floating-point error.

#### 1.4.1 Worked example: $f(t) = \exp(t^2)$

Use the function and evaluation point of Section 1.3.1, now with truncation order  $p = 2$ . Seed  $\widehat{t} = 1 + \varepsilon$  and impose  $\varepsilon^3 = 0$ . The square is obtained by multiplying two OTI numbers  $a = b = 1 + \varepsilon$ :

$$\widehat{v} = \widehat{t}\widehat{t} = 1 + 2\varepsilon + \varepsilon^2.$$

The coefficients of  $\widehat{v}$  are therefore 1, 2, and 1. The next operation is  $g(v) = \exp(v)$ . Its input is the OTI number  $\widehat{v}$ , which is decomposed into a real part and a perturbation expressed in powers of  $\varepsilon$ :

$$\widehat{v} = v_0 + 2\varepsilon + \varepsilon^2, \quad v_0 = 1. \quad (12)$$

The Taylor series expansion (TSE) is taken *about the real part*  $v_0$ . Before substituting the OTI input, write the expansion at a general argument  $s$ :

$$g(s) = g(v_0) + g'(v_0)(s - v_0) + \frac{g''(v_0)}{2!}(s - v_0)^2 + \sum_{j=3}^{\infty} \frac{g^{(j)}(v_0)}{j!}(s - v_0)^j. \quad (13)$$

Now evaluate Equation (13) at the full OTI argument  $s = \widehat{v} = v_0 + 2\varepsilon + \varepsilon^2$ . The displacement from the expansion point is  $(v_0 + 2\varepsilon + \varepsilon^2) - v_0 = 2\varepsilon + \varepsilon^2$ , giving

$$\begin{aligned} g(v_0 + 2\varepsilon + \varepsilon^2) &= g(v_0) + g'(v_0)[(v_0 + 2\varepsilon + \varepsilon^2) - v_0] \\ &\quad + \frac{g''(v_0)}{2!}[(v_0 + 2\varepsilon + \varepsilon^2) - v_0]^2 \\ &= g(v_0) + g'(v_0)(2\varepsilon + \varepsilon^2) + \frac{g''(v_0)}{2!}(2\varepsilon + \varepsilon^2)^2. \end{aligned} \quad (14)$$

Terms of degree three and higher vanish because  $(2\varepsilon + \varepsilon^2)^3 = 0$  in the order-two algebra. Equation (14) is therefore exact in that algebra, rather than an approximation obtained by assuming a small real perturbation. The function and its elementary derivatives are evaluated at the real scalar  $v_0$ ; the powers of the displacement use OTI arithmetic.

For the exponential,  $g(1) = g'(1) = g''(1) = e$ . Substituting these real values and the perturbation from Equation (12) gives

$$\exp(1 + 2\varepsilon + \varepsilon^2) = e + e(2\varepsilon + \varepsilon^2) + \frac{e}{2!}(2\varepsilon + \varepsilon^2)^2.$$

The remaining multiplication is truncated according to  $\varepsilon^3 = 0$ :

$$(2\varepsilon + \varepsilon^2)^2 = 4\varepsilon^2 + 4\varepsilon^3 + \varepsilon^4 = 4\varepsilon^2.$$

Collecting equal powers of  $\varepsilon$  yields the output of the overloaded exponential:

$$\widehat{f} = e + 2e\varepsilon + e\varepsilon^2 + 2e\varepsilon^2 = e + 2e\varepsilon + 3e\varepsilon^2. \quad (15)$$

The exponential's overload uses its known derivatives at the real part; it does not require a manually derived formula for the derivative of the full composition  $f(t) = \exp(t^2)$ . Its coefficients are  $e$ ,  $2e$ , and  $3e$ . Applying Equation (11) gives

$$f(1) = e, \quad f'(1) = 1!(2e) = 2e, \quad f''(1) = 2!(3e) = 6e.$$

The second-order coefficient is  $3e$ , whereas the second derivative is  $6e$ . These results match Equation (4) exactly in the truncated algebra, without choosing a numerical step size. The evaluation trace still consists of a multiplication followed by an exponential; the definitions of those operations propagate the coefficients. Section 1.5.2 performs this same calculation using real matrices.

## 1.5 Cauchy–Riemann matrix representation

Let  $N$  be the  $(p+1) \times (p+1)$  matrix with ones immediately below the diagonal and zeros elsewhere. Since  $N^{p+1} = 0$ , the OTI number defined in Section 1.4 has the representation  $C(a) = \sum_{j=0}^p a_j N^j$ . At order two,

$$N = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad C(a) = \begin{bmatrix} a_0 & 0 & 0 \\ a_1 & a_0 & 0 \\ a_2 & a_1 & a_0 \end{bmatrix}.$$

Multiplying  $C(a)$  by the coefficient vector  $[b_0, b_1, b_2]^T$  produces the coefficients of  $ab$ . Consequently,

$$C(a+b) = C(a) + C(b), \quad C(ab) = C(a)C(b).$$

This is the generalized Cauchy–Riemann representation used here: a real matrix implements OTI multiplication. The convention adopted here uses lower triangular matrices and coefficients ordered by increasing degree. An upper triangular convention would require corresponding changes in extraction.

### 1.5.1 Univariate functions in the representation

Because  $C$  preserves sums and products, it preserves every polynomial and therefore every convergent power series. For an analytic  $f$ ,

$$f(C(a)) = C(f(a)), \quad (16)$$

where the left-hand side is the *matrix* function of  $C(a)$ , evaluated through matrix multiplication, and not  $f$  applied separately to each entry. Evaluating a matrix function of the representation therefore returns the representation of the OTI result, and no OTI scalar type is required.

At first order, seed the input as  $t + \varepsilon$  with  $\varepsilon^2 = 0$ . The real number  $t$  contributes  $tI_2$  and the seed contributes  $N$ , so

$$C(t + \varepsilon) = tI_2 + N = \begin{bmatrix} t & 0 \\ 1 & t \end{bmatrix}.$$

Take  $f = \exp$ . The two terms commute and  $N^2 = 0$ , so the series defining the matrix exponential terminates after two terms:

$$\exp(tI_2 + N) = e^t(I_2 + N) = \begin{bmatrix} e^t & 0 \\ e^t & e^t \end{bmatrix}.$$

Its first column is  $[e^t, e^t]^T$ , which is  $[f(t), f'(t)]$ . Because  $\exp$  is its own derivative, this example on its own cannot separate the method from a coincidence; taking  $f = \sin$  instead gives the first column  $[\sin t, \cos t]^T$ .

The first column carries the answer because  $C(a)$  is lower triangular with constant diagonals: its first column is precisely the coefficients  $a_0, a_1, \dots, a_p$ .

At truncation order  $p$  the same seed gives a  $(p+1) \times (p+1)$  matrix with  $t$  on the diagonal and ones on the first subdiagonal. Applying Equation (16),

$$C(f(t + \varepsilon)) \text{ has first column } \left[ f(t), \frac{f'(t)}{1!}, \frac{f''(t)}{2!}, \dots, \frac{f^{(p)}(t)}{p!} \right]^T. \quad (17)$$

These entries are Taylor coefficients, not derivatives. At first order the distinction is invisible because  $1! = 1$ ; at second order it is not. For  $f = \exp$  at order two,

$$\exp \begin{bmatrix} t & 0 & 0 \\ 1 & t & 0 \\ 0 & 1 & t \end{bmatrix} = e^t \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & 1 & 1 \end{bmatrix},$$

whose first column is  $[e^t, e^t, e^t/2]^T$ . The second derivative follows from the factorial scaling of Equation (11), as  $2!(e^t/2) = e^t$ .

Nothing here is truncated by hand. The power series of  $f$  terminates on its own because  $N^{p+1} = 0$ , and the truncation order is fixed once, by the size of the matrix chosen to carry the seed.



### 1.5.2 Worked example: $f(t) = \exp(t^2)$

Represent the order-two seed  $\hat{t} = 1 + \varepsilon$  from Section 1.4.1 by the ordinary real matrix

$$T = C(\hat{t}) = I_3 + N = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

The same evaluation trace becomes a matrix multiplication followed by a matrix exponential. The first operation gives

$$V = T^2 = I_3 + 2N + N^2 = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Thus  $V = C(1 + 2\varepsilon + \varepsilon^2)$ : its first column contains exactly the intermediate coefficients from Section 1.4.1. For the second operation, write  $V = I_3 + K$  with  $K = 2N + N^2$ . Since  $K^2 = 4N^2$  and  $K^3 = 0$ , the matrix exponential reduces to

$$W = \exp(V) = e \left( I_3 + K + \frac{K^2}{2} \right) = e \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 2 & 1 \end{bmatrix}. \quad (18)$$

The first column of  $W$ , equivalently  $W[1, 0, 0]^T$ , is  $[e, 2e, 3e]^T$ . It contains the Taylor coefficients, so multiplying the third entry by  $2!$  yields the second derivative:

$$f(1) \approx 2.718281828459, \quad f'(1) \approx 5.436563656918, \quad f''(1) \approx 16.309690970754.$$

These are the same results as Equation (15). No OTI scalar type is required: an ordinary real matrix multiplication computes  $T^2$ , and a real matrix-exponential routine computes  $\exp(V)$ . These are matrix operations, not entrywise squaring or exponentiation. The matrix representation changes how the coefficients are stored and propagated; the factorial extraction rule remains the same as in Section 1.4.

### 1.5.3 Matrices of OTI numbers

For a square matrix whose entries are OTI numbers, collect the coefficients of equal degree into ordinary real matrices:

$$M = M_0 + M_1\varepsilon + M_2\varepsilon^2, \quad M_j \in \mathbb{R}^{m \times m}. \quad (19)$$

Specifically, if entry  $(r, s)$  is  $M_{rs} = m_{rs,0} + m_{rs,1}\varepsilon + m_{rs,2}\varepsilon^2$ , then  $(M_j)_{rs} = m_{rs,j}$ . Thus  $M_j$  contains the order- $j$  coefficient of every entry of  $M$ ; its entries are real scalars, not OTI numbers. The corresponding representation is a *block matrix of coefficient matrices*:

$$\mathcal{C}(M) = \begin{bmatrix} M_0 & 0 & 0 \\ M_1 & M_0 & 0 \\ M_2 & M_1 & M_0 \end{bmatrix} \in \mathbb{R}^{3m \times 3m}. \quad (20)$$

Every displayed block in Equation (20), including each zero block, has size  $m \times m$ . The complete block array is an ordinary real  $3m \times 3m$  matrix, suitable for real matrix operations. At truncation order  $p$ , the assembled matrix has size  $(p+1)m \times (p+1)m$ .

It preserves matrix products, including their order. Because the exponential is defined by a convergent series of sums and products, it also preserves the matrix exponential. This lets a real-valued numerical routine evaluate the OTI exponential without requiring a custom scalar type.

## 2 Implementation

### 2.1 Original response calculation

The repository example evaluates a vector response  $F(\Delta t)$  containing ten final voxel temperatures. The differentiation parameter is the duration  $\Delta t$ , nominally 50 s; the arrays  $A, B, e, x_k, u$  remain fixed. The original `MatrixExponentialTest.jl` is retained without modification. Separate entry points, `CTSEExample.jl` and `CRExample.jl`, call reusable functions implementing the methods of Section 1.

The calculation already uses an augmented matrix exponential:

$$H = \begin{bmatrix} A & B & I_n \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad z_0 = \begin{bmatrix} x_k \\ u \\ e \end{bmatrix}, \quad F(t) = P_x \exp(tH) z_0, \quad P_x = [I_n \ 0 \ 0]. \quad (21)$$

Here  $P_x$  selects the first  $n$  entries. The top block row of  $\exp(tH)$  supplies  $A_d, B_d$ , and the integrated forcing  $e_d$ , giving  $F(t) = A_d x_k + B_d u + e_d$ . No additional physical model is required to apply the differentiation methods. The shared `response` function in `src/OTISensitivity.jl` encapsulates this calculation as shown in Listing 1.

```
function response(A, B, e, x, u, dt)
    Ad, Bd, ed = discretize_linear_dynamics(A, B, e, dt)
    return Ad*x + Bd*u + ed
end
```

Listing 1: Reusable response evaluation.

The original final update adds  $e$  rather than  $e_d$ . Its example has  $e = 0$ , so this distinction does not change its result. The shared routine uses  $e_d$  and is tested with nonzero forcing. This correction and the extraction of reusable functions are supporting changes, independent of either differentiation method.

### 2.2 CTSE: complex input and derivative extraction

Applying Section 1.3 requires replacing  $t_0$  by  $t_0 + ih$  in Equation (21). Consequently,

$$\widetilde{M} = (t_0 + ih)H = t_0 H + ihH, \quad (22)$$

$$\widetilde{F} = P_x \exp(\widetilde{M}) z_0. \quad (23)$$

Only the duration is perturbed. The matrix  $H$  and vector  $z_0$  retain their original real values. The scaled matrix becomes complex, while its size remains  $21 \times 21$ . Since  $H$  is constant,

$$\widetilde{F} = F(t_0) + ih P_x H \exp(t_0 H) z_0 + O(h^2),$$

so the coefficient of  $ih$  is the requested derivative  $F'(t_0)$ .

The original routine forms a real matrix  $H$  and scales it in place (Listing 2). A complex timestep cannot be stored in that real array. The shared routine therefore allocates the scaled matrix out of place (Listing 3); its timestep argument accepts `dt::Number`, including complex values.

```
H .*= dt
G = exponential!(H)
```

Listing 2: Matrix scaling and exponential in the original script.

```
G = exponential!(augmented_dynamics(A, B) * dt)
```

Listing 3: Replacement in the shared discretization routine.

`ctse_response` in `src/SensitivityComparison.jl` then passes the complex timestep through the response routine and extracts the result as shown in Listing 4. The example uses  $h = 10^{-20}$  s.

```
z = response(A, B, e, x, u, complex(dt, h))
return (; value=real.(z), derivative=imag.(z)/h)
```

Listing 4: CTSE evaluation and extraction; input validation is omitted from this excerpt.

Here  $\mathbf{z}$  is the complex response  $\tilde{F}$ , not the augmented vector  $z_0$ . The matrix-exponential routine, block extraction, and response multiplication follow the same calculation as the real evaluation. Complex components are preserved until the final real and imaginary extraction in Equation (3); no sensitivity equation is added. `CTSEExample.jl` saves these values and first derivatives to `results/ctse.csv`. Figure 4, upper-left panel, compares its first derivatives with the other methods; Figure 5, upper-left panel, assesses the imaginary step size.

### 2.3 OTI-CR: real block assembly and coefficient extraction

For the method of Sections 1.4 and 1.5, the seeded duration is  $t_0 + \varepsilon$ . At second order, the OTI matrix  $M = (t_0 + \varepsilon)H$  has coefficient matrices  $M_0 = t_0H$ ,  $M_1 = H$ , and  $M_2 = 0$ . Substitution into Equation (20) gives

$$\mathcal{H} = C(t_0 + \varepsilon) \otimes H = \begin{bmatrix} t_0H & 0 & 0 \\ H & t_0H & 0 \\ 0 & H & t_0H \end{bmatrix}. \quad (24)$$

The Kronecker product  $\otimes$  replaces each scalar entry by that scalar times  $H$ . Every block is  $21 \times 21$ ; Julia assembles a single real  $63 \times 63$  matrix. At order  $p$ , its dimension is  $21(p + 1)$ .

Instead of passing  $t_0H$  to the exponential, `oti_response` in `src/OTISensitivity.jl` passes the block representation, as shown in Listing 5. The helper `oti_matrix(seed)` constructs  $C(t_0 + \varepsilon)$  from the coefficients  $[t_0, 1, 0, \dots, 0]$ .

```
H = augmented_dynamics(A, B)
seed = zeros(promote_type(Float64, typeof(dt)), order+1)
seed[1] = dt
order > 0 && (seed[2] = 1)
G = exponential!(kron(oti_matrix(seed), H))
```

Listing 5: OTI seed and real block exponential.

Write  $G_0(t) = \exp(tH)$ . The second-order lifted exponential satisfies

$$\exp(\mathcal{H}) = \begin{bmatrix} G_0 & 0 & 0 \\ G'_0 & G_0 & 0 \\ G''_0/2! & G'_0 & G_0 \end{bmatrix}. \quad (25)$$

The first block column, multiplied by  $z_0$ , contains the response coefficients. The top  $n$  entries of block  $j$  yield  $F^{(j)}(t_0)/j!$ .

Listing 6 shows the corresponding extraction and factorial scaling. Here  $\mathbf{m}$  is the augmented dimension,  $\mathbf{n}$  is the number of response entries, and  $\mathbf{G}$  is the lifted exponential of Equation (25).

```
m = size(H, 1)
z = vcat(x, u, e)
coefficients = hcat([G[j*m+1:j*m+n, 1:m] * z for j in 0:order]...)
derivatives = copy(coefficients)
factor = one(eltype(derivatives))
for j in 1:order
    factor *= j
    derivatives[:, j+1] .*= factor
end
```

Listing 6: Extraction of OTI coefficients and conversion to derivatives.

Column one contains the nominal value. Column  $j + 1$  of **coefficients** contains the order- $j$  Taylor coefficient; the same column of **derivatives** contains the actual derivative. **CRExample.jl** saves orders zero through four to **results/cr.csv**. The scalar type remains real; the changes are the larger matrix passed to the solver and the extraction of its coefficient blocks. Dense storage grows quadratically and typical dense exponential work cubically with the enlarged dimension. This implementation is a baseline for method comparisons rather than an optimized solver.

## 2.4 Comparison and verification code

**CompareSensitivities.jl** runs the unchanged original in an isolated Julia module, runs the implementations in Section 2, and writes comparison CSV files and plots. The original supplies a function value only; it has no derivative implementation. The “FD baseline” function value is the unperturbed real response, not a derivative.

For derivative order  $j = 1, \dots, 4$ , finite differences (FD) use seven real response evaluations:

$$D_j(h) = h^{-j} \sum_{k=-3}^3 w_k^{(j)} F(t + kh), \quad \sum_{k=-3}^3 w_k^{(j)} k^q = \begin{cases} j!, & q = j, \\ 0, & 0 \leq q \leq 6, q \neq j. \end{cases} \quad (26)$$

The moment conditions in Equation (26) make the formula exact for polynomials through degree six. The weights are computed in exact rational arithmetic, then converted to floating point for use with response values. Symmetry gives truncation error  $O(h^6)$  for orders one and two, and  $O(h^4)$  for orders three and four. Small  $h$  increases cancellation and roundoff, roughly as  $\epsilon_{\text{mach}}/h^j$ . The selected step is  $h_j = \max(|t|, 1)\epsilon_{\text{mach}}^{1/(j+a_j)}$  in the code’s seconds convention, where  $a_j$  is six or four respectively. At  $t = 50$  s these steps are approximately 0.290, 0.552, 0.290, and 0.552 s. This is a heuristic balance, not a universal optimal step. Figure 5 presents a sweep of  $h$  to assess its effect.

An independent derivative reference follows by differentiating the augmented response  $F(t) = P_x \exp(tH)z_0$  defined in Section 2.1:

$$F' = AF + Bu + e, \quad F^{(j)} = AF^{(j-1)} \quad (j \geq 2). \quad (27)$$

The reference value  $F$  is computed by diagonalizing the symmetric  $A$  and integrating each scalar mode. The forcing integral is  $(\exp(\lambda t) - 1)/\lambda$ , with limit  $t$  at  $\lambda = 0$ . This handles the insulated model’s zero mode without inverting its singular  $A$ . Errors are measured against Equation (27) and the modal reference value. Absolute errors have units  $\text{K/s}^j$ ; relative errors are  $\|y - y_{\text{ref}}\|_\infty / \|y_{\text{ref}}\|_\infty$  over all ten voxels, avoiding division by an individual near-zero derivative. The comparison also directly checks CTSE against OTI-CR and FD against OTI-CR.

The scripts can be executed from the repository root using:

```
julia --project=. CTSEExample.jl
julia --project=. CRExample.jl
julia --project=. CompareSensitivities.jl
julia --project=. test/runtests.jl
```

The comparison script writes per-voxel values and derivatives, error summaries, and step sweeps to CSV files in **results/**. The figures and table in Section 3 are generated from these evaluations.

### 3 Results

#### 3.1 Function values and derivative agreement

The implementations in Sections 2.2 and 2.3 are evaluated at  $\Delta t = 50$  s using the verification procedure in Section 2.4. Table 2 reports errors against the independent analytical reference. The CTSE and OTI-CR first derivatives have relative errors of approximately  $2.54 \times 10^{-14}$  and  $1.64 \times 10^{-14}$ , respectively. FD agrees through fourth order, with its relative error increasing from  $1.39 \times 10^{-11}$  at first order to  $2.10 \times 10^{-5}$  at fourth order. This reduced accuracy is consistent with the cancellation described in Section 2.4; FD provides corroborating evidence rather than an exact reference.

Table 2: Errors at  $\Delta t = 50$  s relative to the independent analytical reference. Order zero denotes the function value. Absolute errors have units  $\text{K/s}^j$  for derivative order  $j$ ; relative infinity errors are dimensionless.

| Method   | Order | Max. absolute error | Relative infinity error |
|----------|-------|---------------------|-------------------------|
| Original | 0     | 1.78e-11            | 3.03e-14                |
| CTSE     | 0     | 1.38e-11            | 2.34e-14                |
| CTSE     | 1     | 9.81e-14            | 2.54e-14                |
| OTI-CR   | 0     | 1.78e-11            | 3.03e-14                |
| OTI-CR   | 1     | 6.35e-14            | 1.64e-14                |
| OTI-CR   | 2     | 6.80e-15            | 1.16e-13                |
| OTI-CR   | 3     | 9.70e-16            | 3.72e-13                |
| OTI-CR   | 4     | 1.38e-16            | 8.70e-13                |
| FD       | 0     | 1.78e-11            | 3.03e-14                |
| FD       | 1     | 5.37e-11            | 1.39e-11                |
| FD       | 2     | 2.45e-10            | 4.17e-09                |
| FD       | 3     | 1.51e-09            | 5.81e-07                |
| FD       | 4     | 3.33e-09            | 2.10e-05                |

Figure 3 shows agreement of the nominal temperatures across the original script, CTSE, OTI-CR, and the real response used as the FD baseline. Figure 4 compares the corresponding voxel sensitivities; CTSE appears only in the first-order panel.

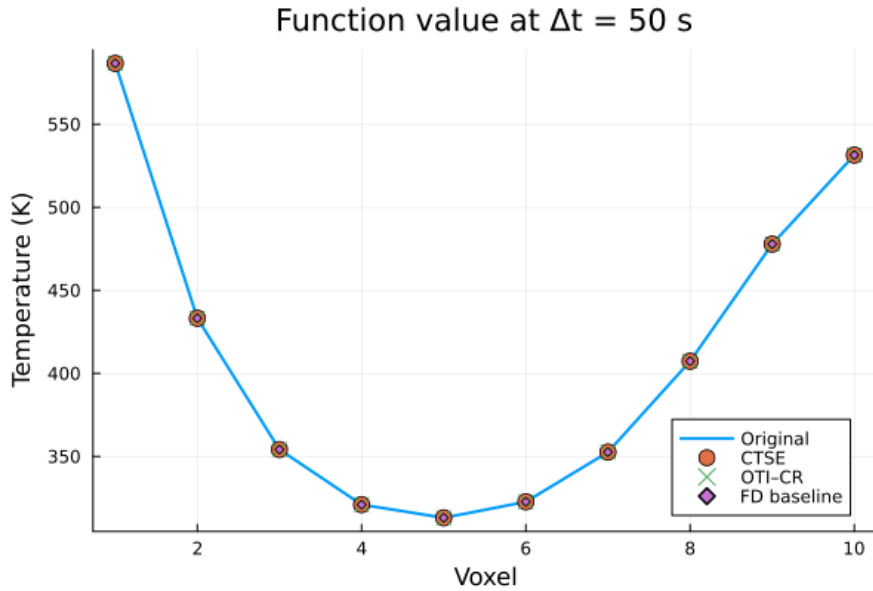


Figure 3: Nominal voxel temperatures at  $\Delta t = 50$  s. CTSE and OTI-CR agree with the unchanged original script and the unperturbed real FD baseline.

All 246 test assertions pass. The tests cover scalar derivatives, OTI algebra, zero duration, nonzero forcing, CTSE steps from  $10^{-8}$  to  $10^{-30}$  s, and FD polynomial exactness.

### 3.2 Derivative profiles and step-size dependence

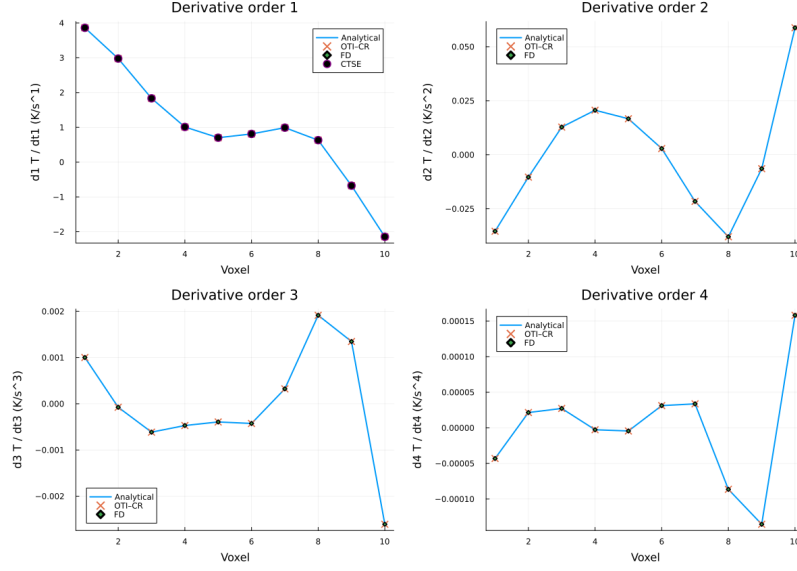


Figure 4: Voxel temperature derivatives at  $\Delta t = 50$  s. The upper-left panel compares CTSE, OTI-CR, FD, and the analytical first derivative. CTSE is omitted from orders two through four.

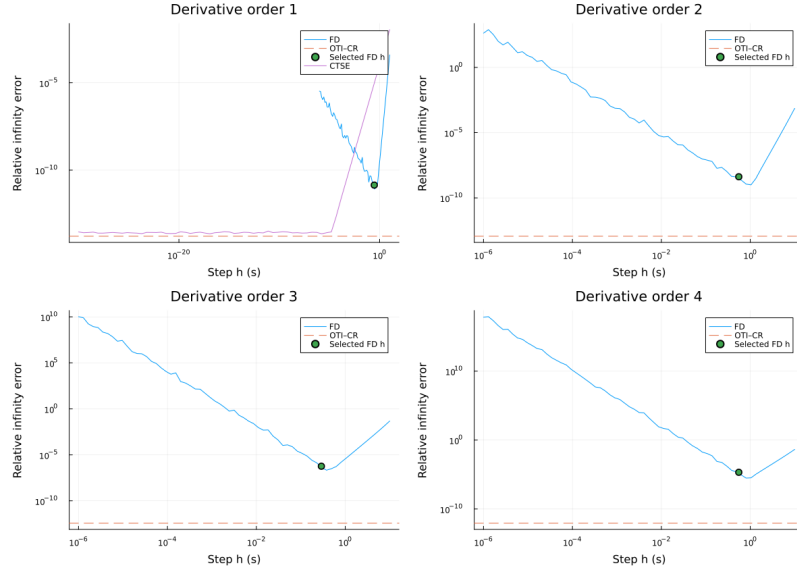


Figure 5: Relative derivative errors versus step size. The upper-left panel compares the CTSE imaginary-step sweep with real finite differences (FD). OTI-CR has no step parameter and is shown as a horizontal reference. Higher-order panels show FD and OTI-CR only.

Figure 5 shows FD accuracy deteriorating for both small and large steps. CTSE maintains first-derivative accuracy for very small imaginary steps in this example. OTI-CR has no step parameter and provides a horizontal error reference. Errors in Figure 5 are floored at machine epsilon only for logarithmic display; the CSV files retain the raw errors.

## References

- [1] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018). [Automatic Differentiation in Machine Learning: a Survey](#). *Journal of Machine Learning Research*, 18(153), 1–43.
- [2] C. H. Bischof, H. M. Bücker, P. Hovland, U. Naumann, and J. Utke (eds., 2008). [Advances in Automatic Differentiation](#). Springer.
- [3] A. Griewank and A. Walther (2008). [Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation](#). Second edition, SIAM.
- [4] MDO Lab. [A Guide to the Complex-Step Derivative Approximation](#).
- [5] J. R. R. A. Martins, P. Sturdza, and J. J. Alonso (2003). [The Complex-Step Derivative Approximation](#).
- [6] M. Aristizabal Cano (2020). [Order truncated imaginary algebra for computation of multivariable high-order derivatives in finite element analysis](#).